

1. Why is Feature Selection Important?

- Removes unnecessary columns
- Reduces complexity
- Improves prediction accuracy
- Speeds up training

```
#Feature Selection
X = df[['bedrooms',
        'bathrooms',
        'square_feet',
        'lot_size_sqft',
        'year_built',
        'garage_spaces',
        'distance_to_city_center_km',
        'crime_rate_index',
        'nearby_school_rating',
        'days_on_market']]

y = df['house_price']

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

Explanation:

"I split the dataset into 80% training data and 20% testing data. The training data teaches the model patterns, while the testing data evaluates how well the model performs on unseen data."

Selected Features

The six most important features selected were:

- Bedrooms
- Bathrooms
- Square Feet
- Lot Size (sqft)
- Property Condition
- Distance to City Center (km)

These features showed the strongest relationship with the target variable and were used for model training.

2. Train-Test Split

After feature selection:

```

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

```

The training data was used to train the models, while the testing data was reserved for evaluating their predictive performance.

3. Model Creation:

Objective

Three regression algorithms were trained to predict house prices. Comparing multiple models helps identify the algorithm that provides the best predictive performance.

Model 1 – Linear Regression

Linear Regression establishes a linear relationship between the selected features and the house price.

```

#Model 1 Linear regression:
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

lr = LinearRegression()
lr.fit(X_train, y_train)

lr_pred = lr.predict(X_test)

print("Linear Regression")
print("MAE :", mean_absolute_error(y_test, lr_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, lr_pred)))
print("R2 :", r2_score(y_test, lr_pred))

```

```

Linear Regression
MAE : 429286.3143270623
RMSE: 518580.50304179726
R2 : 0.036539498163664974

```

Model 2 – Decision Tree Regressor

Decision Tree Regressor predicts the target value by splitting the data into decision nodes based on feature values.

```
#Model 2:
from sklearn.tree import DecisionTreeRegressor

dt = DecisionTreeRegressor(random_state=42)
dt.fit(X_train, y_train)

dt_pred = dt.predict(X_test)

print("Decision Tree")
print("MAE :", mean_absolute_error(y_test, dt_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, dt_pred)))
print("R2 :", r2_score(y_test, dt_pred))
```

```
Decision Tree
MAE : 577410.6666666666
RMSE: 748399.5721983456
R2 : -1.0066360339614566
```

Model 3 – Random Forest Regressor

Random Forest is an ensemble learning algorithm that combines multiple decision trees to improve prediction accuracy.

```
#Model 3
from sklearn.ensemble import RandomForestRegressor

rf = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

rf.fit(X_train, y_train)

rf_pred = rf.predict(X_test)

print("Random Forest")
print("MAE :", mean_absolute_error(y_test, rf_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, rf_pred)))
print("R2 :", r2_score(y_test, rf_pred))
```

```
Random Forest
MAE : 431474.07
RMSE: 534674.9877807919
R2 : -0.02419177069095868
```

Model Comparison:

```
In [31]: import numpy as np
import pandas as pd

results = pd.DataFrame({
    "Model": ["Linear Regression", "Decision Tree", "Random Forest"],
    "MAE": [
        mean_absolute_error(y_test, lr_pred),
        mean_absolute_error(y_test, dt_pred),
        mean_absolute_error(y_test, rf_pred)
    ],
    "RMSE": [
        np.sqrt(mean_squared_error(y_test, lr_pred)),
        np.sqrt(mean_squared_error(y_test, dt_pred)),
        np.sqrt(mean_squared_error(y_test, rf_pred))
    ],
    "R2 Score": [
        r2_score(y_test, lr_pred),
        r2_score(y_test, dt_pred),
        r2_score(y_test, rf_pred)
    ]
})

print(results)
```

	Model	MAE	RMSE	R2 Score
0	Linear Regression	429286.314327	518580.503042	0.036539
1	Decision Tree	577410.666667	748399.572198	-1.006636
2	Random Forest	431474.070000	534674.987781	-0.024192

4. Prediction:

```
best_model = lr
```

```
y_pred = best_model.predict(X_test)
```

"After training, I used the testing dataset to predict house prices. These predictions are compared with the actual prices to evaluate model performance."

5. Model Evaluation:

```
In [34]: #Model Evaluate
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MAE :", mae)
print("MSE :", mse)
print("RMSE :", rmse)
print("R2 Score :", r2)
```

```
MAE : 429286.3143270623
MSE : 268925738135.08353
RMSE : 518580.50304179726
```

```
from sklearn.metrics import r2_score, mean_absolute_error

print("R2 Score:", r2_score(y_test, y_pred))
print("MAE:", mean_absolute_error(y_test, y_pred))
```

Among the three algorithms, Linear Regression achieved the best performance.

Reasons:

- Lowest Mean Absolute Error (MAE)
- Lowest Root Mean Squared Error (RMSE)
- Highest R^2 Score

Although the R^2 value is relatively low, Linear Regression performed better than the Decision Tree and Random Forest models for this dataset.

Therefore, Linear Regression was selected as the final model.